A1 - SHORT REPORT ON CNN EXPERIMENTS USING FASHION-MNIST

Group: Kelompok Berkah

1. Putra Alfa Juliano Bunga (672024212)
2. Nathan Elden Mandeij (672024221)
3. Kristopher Sardo Sihotang (672024234)

1. Introduction

This experiment aims to evaluate how different Convolutional Neural Network (CNN) configurations affect image classification performance on the **Fashion-MNIST** dataset. Four scenarios, V1 to V4, were compared by modifying the CNN architecture and training configuration.

The experiment focuses on two main modifications:

1. **Changing the dataset** from MNIST to Fashion-MNIST.
2. **Modifying the CNN configuration**, including model architecture, Batch Normalization, Dropout, optimizer, learning rate, and activation function.

2. Change 1 - Dataset: MNIST to Fashion-MNIST

The original hands-on notebook trained a CNN to classify handwritten digits (0–9) from the MNIST dataset. We replaced this with Fashion-MNIST, which contains 28×28 grayscale images of clothing items across 10 categories: T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, and Ankle boot.

Aside from swapping the dataset loader (tensorflow.keras.datasets.mnist → fashion_mnist) and adding the class-name list for readable predictions, the preprocessing pipeline (reshaping to 28×28×1, normalizing pixel values to [0, 1], and one-hot encoding the labels) and the baseline CNN architecture were kept identical to the original material, so that any difference in results could be attributed to the dataset itself rather than to the pipeline.

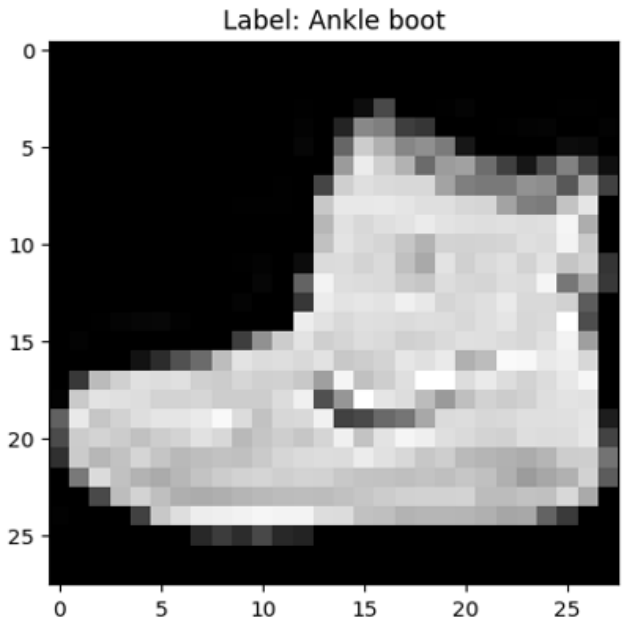


Figure 1. Sample image from the Fashion-MNIST training set with its label.

Fashion-MNIST is widely regarded as a more challenging benchmark than MNIST because clothing categories share more visual similarity with one another (for example, Shirt, Pullover, and Coat) than handwritten digits do, so we expected overall accuracy to be noticeably lower than the ~99% typically achieved on MNIST.

3. Change 2 - CNN Configuration Experiments

Using the Fashion-MNIST baseline as the reference point (v1), we designed three additional configurations that each isolate a different aspect of the network so that their individual effects could be observed:

Run	Theme	Changes from baseline
v1	Baseline	None (identical to the original hands-on model)
v2	Model capacity	Larger conv filters (32→64, 64→128) plus one extra conv block (128, padding="same"), BatchNormalization after every Conv2D layer, and the Dense layer widened from 128 to 256 units.

v3	Training dynamics	Dropout reduced from 0.5 to 0.3; optimizer switched from Adam to RMSprop with a lower learning rate (0.0001); trained for only 5 epochs.
v4	Combination + fine-tuning	Combines the v2 architecture with the v3 regularization/optimizer settings, then adds further tuning: epochs increased to 8 and ReLU replaced with ELU activation throughout (batch size was kept at 128 in the executed run).

4. Model Evaluation Results

Scenario	Test Accuracy	Test Loss	Epochs	Batch Size
V1: Baseline	90.47%	0.2632	10	128
V2: Model capacity	90.02%	0.3162	10	128
V3: Training Dynamic	83.50%	0.4544	5	128
V4: Combination	90.66%	0.2775	8	128

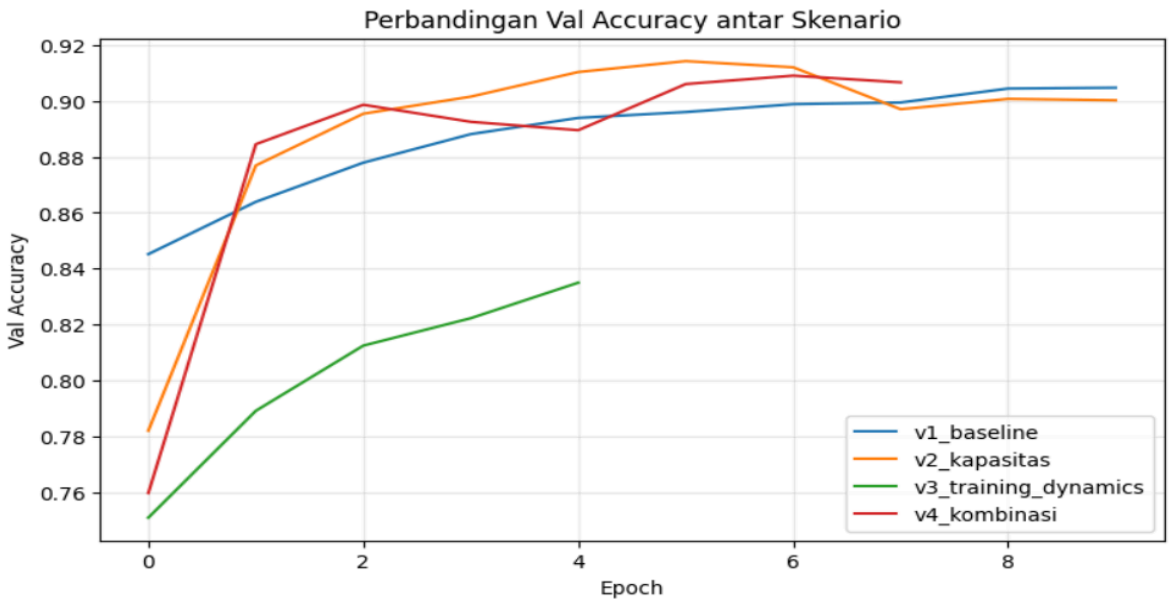


Figure 2. Validation accuracy per epoch for all four scenarios (v1 baseline, v2 model capacity, v3 training dynamics, v4 combination).

5. Lessons Learned

Increasing Capacity is Not a Guarantee: Scenario v2 demonstrated that merely adding layers and neurons without balancing other hyperparameters can trigger overfitting, as evidenced by the higher test loss (0.3162) compared to the baseline. The Importance of Balancing Epochs & **Learning Rate:** Scenario v3 resulted in the lowest accuracy (83.50%). This occurred because pairing a very low learning rate with a short training duration (5 epochs) prevented the model from reaching optimal convergence (underfitting).

Synergy of Architecture Components: The best performance (90.66% accuracy) was achieved in scenario v4, proving that deeper models can generalize well when supported by the right activation function (ELU), Batch Normalization, and carefully calibrated optimizers and regularization techniques. Nevertheless, the baseline model proved to be highly efficient and competitive for this dataset. •

6. Conclusion

Switching the dataset from MNIST to Fashion-MNIST confirmed that the difficulty of the classification task, not just the model architecture, has a large effect on achievable accuracy. Across the four CNN configurations we tested on Fashion-MNIST, the combined configuration (v4) achieved the best test accuracy (90.66%), narrowly ahead of the baseline (90.47%) and the larger capacity-only model (90.02%), while the reduced-dropout/low-learning-rate configuration trained for fewer epochs (v3) performed the worst (83.50%) mainly due to insufficient training time. This suggests that, for this dataset and network family, moderate architectural capacity paired with careful regularization and optimizer tuning is more effective than increasing model size alone.